

1 Explainability in Reinforcement Learning

2 Maxime Alaarabiou

3 Saturday 25th July, 2026

Chapter 1

Explainability in Reinforcement Learning

Today, the majority of Artificial Intelligence (AI) agents with which humans interact are deployed in controlled or low-risk environments. For example, agents capable of playing games such as Go operate in settings where potential errors remain limited in terms of consequences [Silver et al., 2017]. Even more autonomous systems, such as agents based on Large Language Models (LLMs), remain constrained by human supervision mechanisms and are often restricted to tasks such as code generation or information retrieval [Chen et al., 2021, Lewis et al., 2021]. Safety-critical applications integrating machine learning components are still rare. Introducing autonomous agents in such environments, requires the establishment of strong guarantees [Seshia et al., 2022]. These requirements include in particular:

- validating the system’s functioning before deployment,
- identifying the origin of a malfunction in case of failure,
- and certifying that an observed failure will not occur again.

The main challenge lies in the fact that the performing agents currently available rely on both Reinforcement Learning (RL) and Deep Learning (DL) [Mnih et al., 2013]. As a result, the mechanisms underlying their decisions cannot be understood directly from the learned model. This lack of explainability makes their use difficult in safety-critical contexts. To address this limitation, a research field has emerged: eXplainable Reinforcement Learning (XRL), a subfield of eXplainable Artificial Intelligence (XAI). The objective of XRL is to develop explainability methods that make it possible to obtain explanations of RL agents.

This chapter presents the main existing XRL methods. We first define what is meant by an explanation in the context of XAI. We then propose a taxonomy for organizing existing XRL methods. This taxonomy serves as the basis for the state-of-the-art review presented in the remainder of the chapter.

1.1 Explanation in Artificial Intelligence

In this section, we define what an explanation is in the context of XAI and how it can be obtained. We first introduce the concept of explanation in general, along with the explainability method used to produce it. We then describe how this concept applies to XAI. Finally, we further detail the properties that characterize an explainability method in the context of XAI.

1.1.1 Explanation

Establishing what is meant by an explanation is challenging due to the fundamentally multidisciplinary nature of the concept. When we refer to explanation, we are inherently referring to an explanation provided to a human. As soon as humans are involved, the problem is no longer purely computational or mathematical. It involves several fields of research, such as philosophy, psychology, and sociology. So far, these disciplines have not converged on a unified definition of explanation that can be directly applied to XAI [Miller, 2019, Bellucci et al., 2021].

For this reason, we propose the following definition of explanation. An explanation involves two entities: the object (Definition 1.1.1) being explained and the receiver of the explanation. The receiver is a human who aims to understand how the object operates and will be referred to as the observer throughout this manuscript (Definition 1.1.2). The observer seeks to understand the object and, in doing so, builds a mental model of its functioning. An explanation is therefore what allows the observer to refine and correct their mental model of how the object operates (Definition 1.1.4). This refinement is obtained through a transformation process that produces an explanation from the object being explained. We refer to this process as an explainability method (Definition 1.1.5).

For an observer to understand an explanation, the information it contains must be interpretable (Definition 1.1.3). Information is considered interpretable when it can be directly understood by a human without requiring any additional transformation process.

Definition 1.1.1: Object

The object of an explanation is the system the observer wants to understand.

Definition 1.1.2: Observer

The observer is the human who aims to understand an object. The observer possesses a mental model of the object being explained. This mental model is an internal representation of how the observer believes the system operates. This representation may be incomplete or inaccurate.

Definition 1.1.3: Interpretable

Information is interpretable when it can be directly understood by an observer without requiring any additional transformation process.

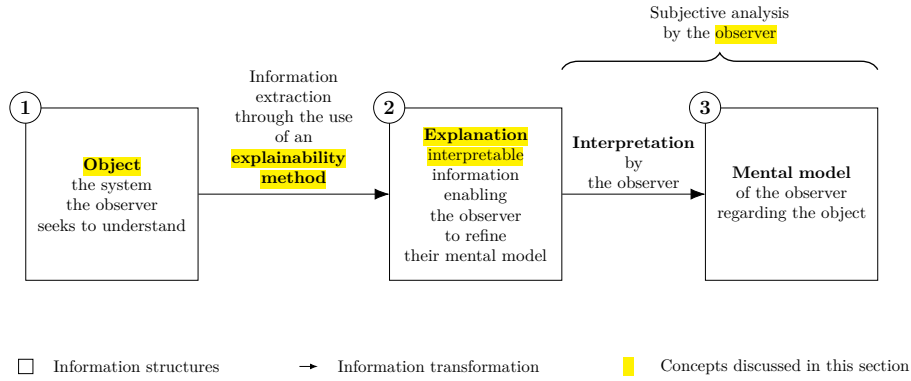


Figure 1.1: Explanation process in explainable artificial intelligence.

Definition 1.1.4: Explanation

An explanation is any interpretable information that enables an observer to refine or correct their mental model of the object of the explanation.

Definition 1.1.5: Explainability Method

An explainability method is a transformation process that produces an explanation from the object being explained.

This definition of explanation provides the foundation for the approach to explainability adopted throughout this manuscript. Its generality allow it to encompass the different explainability methods considered in the following sections.

Moreover, it avoids a binary interpretation of explanation, where a system is considered either explainable or not explainable. From our perspective, the notion of non-explainable system is not meaningful. Considering a system as impossible to explain would therefore contradict the general objective of scientific understanding.

This quantitative perspective allows us to express an intuitive idea: not all explanations provide the same level of value [Doshi-Velez and Kim, 2017]. The effectiveness of an explanation depends on its ability to improve the observer's understanding of the system. For example, describing a marginal aspect of a system is different from correcting an observer's misunderstanding of a key mechanism.

Figure 1.1 summarizes this whole explanation process, from the object of the explanation to the mental model refined by the observer, as a chain of three information structures, depicted as boxes and connected by arrows denoting a transformation of information:

- (1) The **object** of the explanation, that is, the system the observer seeks to understand.
- (2) The **explanation** itself, the interpretable information produced from the object. This transformation is labeled information extraction, and is

performed through the use of an explainability method (Definition 1.1.5), detailed further in Section 1.1.3.

- (3) The **mental model** held by the observer regarding the object. This transformation is labeled interpretation by the observer, corresponding to the cognitive step through which the observer refines their mental model upon receiving the explanation.

A brace spanning boxes (2) and (3) indicates that this second half of the process, from the explanation to the resulting mental model, constitutes a subjective analysis carried out by the observer.

1.1.2 Explainable Artificial Intelligence

Now that the general framework of explanation has been established, we can examine how this concept applies in the context of AI. In the context of XAI, the object of explanation is an AI system. More precisely, XAI emerged as a response to the success of DL models [Gunning and Aha, 2019, Barredo Arrieta et al., 2020]. These models achieve high performance through connectionist architectures based on deep Neural Networks (NNs). In these architectures, information is represented in a distributed manner across the network through patterns of activation and connections between computational units. Unlike symbolic approaches, individual components of NNs cannot be directly associated with human-defined concepts. From this starting point, we identify two types of approaches:

- **Substitutive explainability:** In this approach, the underlying assumption is that there is nothing to explain within a NN. There is no semantic meaning associated with each computational component, and therefore nothing to interpret. This corresponds to the so-called “black box” view of NNs. Under this assumption, XAI consists of replacing as much of the DL system as possible with alternative AI methods [Rudin, 2019].
- **Intrinsic explainability:** In this approach, it is argued that NNs inherently contain mechanisms that can be explained. The limitation is considered to lie not in the networks themselves, but in the absence of adequate tools to analyze their functioning [Olah et al., 2020].

This manuscript considers methods from both approaches in order to provide a comprehensive overview of the XAI field. According to the definition of explanation adopted in this manuscript (Definition 1.1.4), we consider that any system can be explained. As a consequence, the perspective adopted throughout this manuscript is that of intrinsic explainability.

1.1.3 Explainability Method in XAI

Now that the general framework of XAI has been established, we need to further detail the properties of an explanation in this context. In our framework, an explanation in the field of XAI can be characterized by three components [Guidotti et al., 2018]:

- **Source:** refers to the origin of the information used by the explainability method. The explainability method transforms the raw information produced by the AI system into an explanation that can be understood by an observer.

- 134 • **Form:** refers to the representation through which the explanation is
135 communicated. The form defines how the information is presented to
136 the observer and must allow human interpretation. An explanation can
137 take many different forms, including text, images, computer code, and
138 mathematical equations.
- 139 • **Subject:** refers to the aspect of the AI system that is being explained. In
140 XAI, this distinction is commonly associated with the difference between
141 local and global explanations. A local explanation describes why an AI
142 system produced a specific output in a given situation. A global explanation
143 aims to describe the general functioning of the AI system.

144 These three aspects of an explanation are entirely determined by the ex-
145 plainability method (Definition 1.1.5) used to produce it. In this context, an
146 explainability method is a transformation process that maps a source of informa-
147 tion into an explanation characterized by a specific form and subject. Research
148 in XAI focuses on developing new methods capable of performing this trans-
149 formation in order to overcome the lack of interpretability associated with DL
150 models.

151 1.2 Taxonomy of XRL Methods

152 Now that the concept of XAI has been defined, we turn to its application in RL,
153 known as XRL. As in XAI, explanations in XRL are produced by explainability
154 methods and can be described in terms of their subject, source, and form. This
155 section reviews the possible variations of these three aspects in the specific
156 context of RL.

157 1.2.1 Subject

158 The first question to address is the subject of explanation in the context of RL.
159 To answer this question, it is necessary to identify the different components
160 involved in a RL system and determine which of them constitutes the object of
161 explanation. A RL system can be described through two main components: the
162 environment and the policy.

163 The environment defines the dynamics in which the agent evolves, including
164 the possible states, observations, actions, and transitions. Environments are
165 simulations designed by humans, and their dynamics are explicitly defined.
166 Therefore, it is assumed that the environment is known to the observer and does
167 not require explanation.

168 The main object of interest is therefore the agent’s policy. The policy
169 represents the decision-making mechanism of the agent. It is a function that maps
170 observations to actions and is learned through interaction with the environment in
171 order to maximize the expected cumulative reward. The policy is the component
172 that encodes the learning process and that XRL aims to explain.

173 Explaining a policy introduces additional challenges compared with classical
174 XAI settings. In traditional XAI problems, the objective is to explain an isolated
175 decision, such as determining which features of an image led to a classification
176 result. In contrast, RL introduces a temporal dimension into the decision-making
177 process. An action is not only determined by the current observation but also

178 influences future states and rewards. Therefore, understanding the behavior of
 179 an agent may require considering a sequence of interactions between the agent
 180 and its environment.

181 This temporal dimension leads to a distinction between two objectives of
 182 explanation in XRL:

- 183 • **Decision explanations:** focus on identifying the elements of the current
 184 observation that influenced the action selected by the policy. This type of
 185 explanation is close to classical XAI approaches, as it analyzes the rela-
 186 tionship between inputs and outputs while abstracting away the sequential
 187 nature of the decision process.
- 188 • **Strategy explanations:** aim to characterize the agent’s behavior over
 189 time. They focus on sequences of decisions and interactions with the
 190 environment in order to understand how the agent achieves its long-term
 191 objective. This type of explanation is specific to RL, as it addresses the
 192 temporal and goal-oriented nature of the learning process.

193 These two objectives should not be considered as strictly separated categories,
 194 but rather as two extremes of a continuum. Indeed, understanding the elements
 195 of an observation that influence a specific decision can help the observer generalize
 196 this information to infer broader aspects of the agent’s behavior. Conversely,
 197 understanding the agent’s overall strategy can provide insights into the reasoning
 198 behind individual decisions.

199 1.2.2 Source

200 We identify three sources of explanation in RL, all obtained during or after the
 201 learning process and containing information about the agent:

- 202 • **Policies:** policies are functions that generate a distribution over the action
 203 space for a given observation. The objective of RL is to learn a policy that
 204 maximizes the expected cumulative future rewards. During the learning
 205 phase, the policy is iteratively improved through interactions with the
 206 environment.
- 207 • **Trajectories:** trajectories represent sequences of successive interactions
 208 between the agent and the environment. At each step, the agent selects
 209 an action based on its observation, which modifies the environment and
 210 produces a reward as well as a new observation. This process continues
 211 until the end of the trajectory.
- 212 • **Value functions:** value functions provide predictions about the future
 213 outcome of a trajectory from a given observation. The most commonly
 214 used value function is the critic function, which estimates the expected
 215 sum of future rewards. This prediction guides the learning process, as the
 216 agent updates its policy to maximize the estimated future return. However,
 217 other types of value functions can also be used as sources of explainability.

218 1.2.3 Form

219 We identify six forms of explanation in the RL setting:

Explanation Subject	Explanation Form	Explainability Method Family	Source of Explanation		
			Policy	Trajectories	Value functions
Decision	Policy Model	<i>Interpretable policy learning</i>	✓		
		<i>Policy approximation via interpretable model</i>	✓		
	Saliency Map	<i>Observation perturbation</i>	✓		
		<i>Observation gradient analysis</i>	✓		
		<i>Attention-based architecture</i>	✓		
	Counterfactual	<i>Latent space usage for counterfactual generation</i>	✓		
Strategy	Agent Trajectory	<i>Hierarchical decomposition</i>	✓	✓	
		<i>Extraction via critic analysis</i>		✓	✓
	Trajectory Prediction	<i>Critic enrichment</i>			✓
	Interaction Model	<i>Bayesian network learning</i>		✓	
		<i>Latent space usage for MDP generation</i>	✓		

Table 1.1: Taxonomy of explainability methods in reinforcement learning.

- **Policy Model:** an interpretable model of the agent’s policy function.
- **Saliency Map:** a weighting of the input features representing their influence on the output of the policy function.
- **Counterfactual:** a modified observation obtained by perturbing the original observation in order to produce a different action from the agent.
- **Agent Trajectory:** a sequence of successive interactions between the agent and the environment.
- **Trajectory Prediction:** a prediction about the future evolution of the trajectory produced by a value function.
- **Interaction Model:** an interpretable model of the interactions between the agent and the environment.

Among the three components characterizing an explanation, the form is the one that varies most significantly across explainability methods. While subjects and sources tend to be relatively stable in the XRL literature, the diversity in how explanations are presented is considerably greater. The list above is by no means definitive and may evolve as new approaches emerge. For this reason, the following section is organized according to the form of explanations and presents existing explainability methods based on this criterion.

1.3 State-of-the-Art Review of XRL Methods

The purpose of this section is to detail the functioning of existing explainability methods (see Table 1.1). They are presented according to the form of explanation they produce, following the taxonomy introduced in Section 1.2.

Each subsection introduces one of the six forms of explanation identified in Section 1.2.3. Within each subsection, each subsubsection is dedicated to a family of methods capable of producing that form of explanation.

A family of methods is defined as a set of methods that share the same source, form, and subject of explanation, as well as a similar procedure for generating explanations. Methods belonging to the same family may differ in their implementation or in the details of the generation process, while relying on the same underlying principle.

1.3.1 Policy Model

The policy function can be represented by a wide variety of function classes. In modern RL, NNs is the most commonly used models for representing policies. Although these models achieve the highest performance, their connectionist nature makes their internal representations non-interpretable.

For this reason, a substitutive approach to XRL aims to replace these models with alternative models considered interpretable. Such models can be understood through direct analysis of their representations. The main classes of interpretable models used to represent policy functions include decision trees [Topin et al., 2021], algorithms [Trivedi et al., 2022], formal logic [Payani and Fekri, 2019], and linear models [Garreau and von Luxburg, 2020].

Explanations derived from an interpretable model primarily provide insights into the decision-making process, since what is modeled is the mapping between an observation and an action. If the model is sufficiently simple or well structured, the observer can form a mental representation of a sequence of decisions across the environment and thus obtain information about the agent’s strategy.

The main limitation of these methods is that interpretable models rely on symbolic representations. When the policy to be represented is too complex, achieving comparable performance requires a large number of rules or conditions. The resulting model can quickly become difficult to analyze and lose its interpretability.

For example, representing a policy capable of playing Go using a symbolic model would require considering a very large number of possible situations and specific cases. Even if such a representation were possible, the resulting model would likely become intractable for human analysis due to its complexity. This illustrates why NN-based approaches have become dominant in such domains, as they can learn complex decision strategies without requiring an explicit enumeration of rules.

To obtain an interpretable policy model, two families of explainability methods are available: *interpretable policy learning* and *policy approximation via interpretable model*.

Interpretable Policy Learning

This family of methods aims at learning a policy represented by an **interpretable** model [Topin et al., 2021, Trivedi et al., 2022]. It is the most direct approach to addressing the challenges posed by XRL. This type of training requires modifying the Markov Decision Processes (MDP) in order to adapt it to the search space and to the type of policy representation. Moreover, this search space often needs to be reduced to make learning feasible. Such a **reduction** requires the

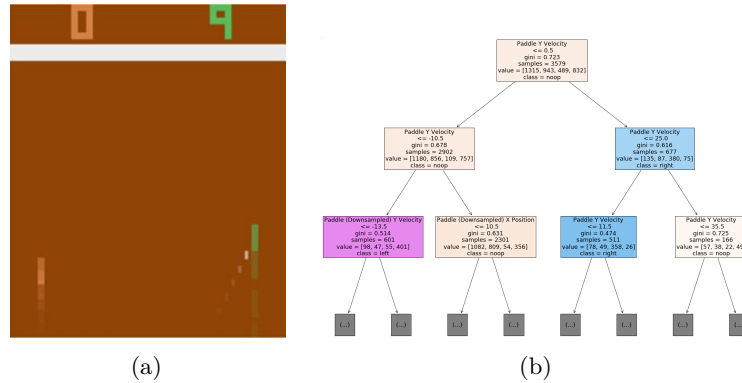


Figure 1.2: Policy approximation via an interpretable model [Sieusahai and Guzdial, 2021]. (a) shows an observation from the Atari game “Pong”, and (b) shows the decision tree obtained by approximating a deep reinforcement learning policy trained on this environment.

integration of **expert knowledge** into the learning process. With this family of methods, the *interpretable policy function* itself is used to provide explanations to the observer.

Policy Approximation via Interpretable Model

An interpretable model can be trained to approximate a policy learned through deep RL [Sieusahai and Guzdial, 2021]. In this setting, the problem is formulated as a supervised learning task, where the original deep RL policy is used to label each observation with the action it selects (see Figure 1.2a). Using this dataset, the interpretable model is trained to reproduce the behavior of the original policy, yielding for instance the decision tree illustrated in Figure 1.2b. The resulting interpretable model is then used as an explanation for the observer. If this approximation is sufficiently faithful, it can also be deployed in the environment in place of the original policy, thereby providing a high degree of control over the agent.

1.3.2 Saliency Map

The influence of each part of the input on the action selected by the policy can be visualized using heatmaps (Figure 1.3). In the RL setting, the input corresponds to the observation received by the agent at a given time step. Heatmaps therefore highlight the regions of the observation that have the greatest influence on the action selection process. By providing this information, they allow the observer to identify which elements of the environment are considered important by the policy when making a decision.

A limitation of heatmap-based methods is that the resulting explanations are not always directly meaningful to the observer. As illustrated in Figure 1.3b, interpreting which regions are important often requires human reasoning and involves a degree of subjectivity. This limitation becomes particularly relevant in RL, where the objective is not only to explain a single decision but also to understand the factors influencing a sequence of decisions over time.

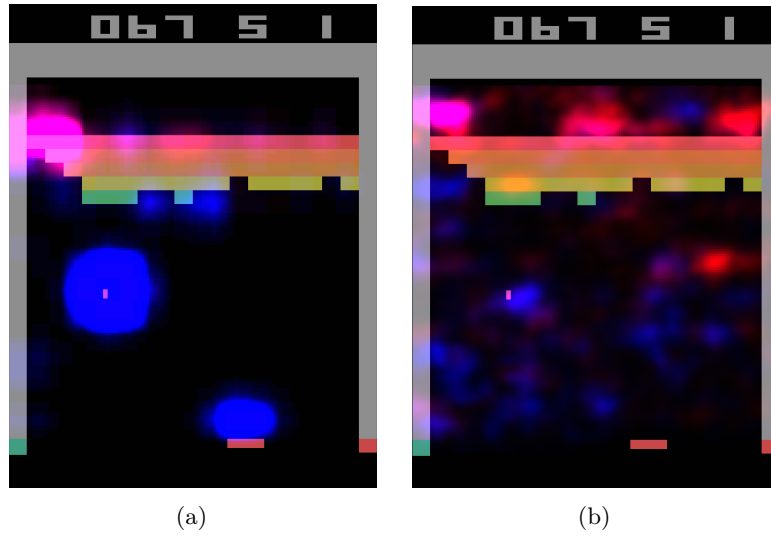


Figure 1.3: Heatmaps obtained by [Greydanus et al., 2018] for the same observation of the Atari game “Breakout”. Highlighted areas correspond to the most influential regions for the policy function’s decision. (a) is obtained via observation perturbation (see Section 1.3.2), whereas (b) is obtained via observation gradient analysis (see Section 1.3.2).

Three families of explainability methods can be used to obtain this type of explanation: *observation perturbation*, *observation gradient analysis*, and *attention-based architectures*.

Observation Perturbation

Perturbation analysis consists in applying various **modifications** to the observation in order to study their **impact** on the output of the **policy function** [Greydanus et al., 2018]. The greater the variation in the output caused by a perturbation applied to a given component of the input vector, the higher the **weight** assigned to that component. The magnitude of the perturbation is measured by computing the distance between the original output and the output obtained from the perturbed input. Using this set of weights, a *heatmap* is constructed, enabling the observer to visualize which parts of the observation vector are most sensitive in the action-selection process (see Figure 1.3a).

Observation Gradient Analysis

Observation **gradient** analysis refers to a set of methods that require a differentiable policy function in order to extract information from it [Simonyan et al., 2014, Weitekamp et al., 2019, Huber et al., 2019]. To this end, the gradient of the policy function is computed with respect to all **components of the input vector**. For each input component, the resulting gradient provides a weighting that represents its importance in the output of the policy function. Based on these weightings, a *heatmap* is constructed to allow the observer to visually identify the parts of the input vector that are the most influential in the policy function’s

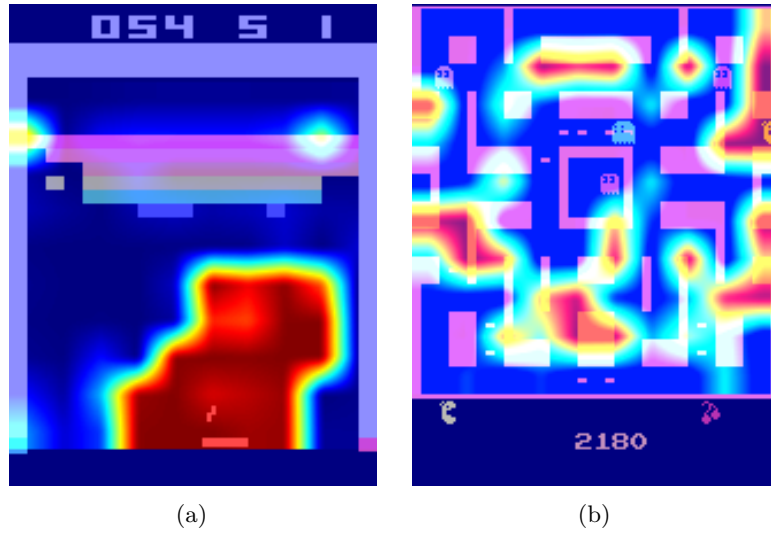


Figure 1.4: Attention-based heatmaps obtained by [Mott et al., 2019] for the Atari games “Breakout” and “Ms. Pac-Man” (see Section 1.3.2). In (a), the highlighted regions are easily interpretable, whereas in (b), the attention is spread across many regions of the observation, making the explanation much harder to interpret.

output (see Figure 1.3b).

Attention-Based Architecture

Attention blocks constitute a NN architecture that makes it possible to weight the relative importance of one part of a vector with respect to the other parts of the same vector [Vaswani et al., 2023]. These weights are computed as a function of the vector itself and are learned by the NN during training. For a given observation, each component of the observation vector is assigned a scalar value. It can therefore be assumed that the components associated with the highest weights are the most influential in determining the output [Mott et al., 2019, Itaya et al., 2021]. This attention-based *heatmap* is then provided to the observer as the explanation (see Figure 1.4a). However, as shown in Figure 1.4b, the attention weights are not always concentrated on a small, easily interpretable set of regions, which can make this type of explanation difficult to exploit.

1.3.3 Counterfactual

In the context of RL, counterfactual explanations are used to generate new observations. This new observation, distinct from the original one, leads the agent to adopt an alternative behavior. A counterfactual explanation aims to explain the agent’s decision-making process. Modifying the observation informs the observer about the adjustments required for the agent to adopt a different behavior. The family of methods available to generate counterfactuals is the *use of the Latent Space (LS) for counterfactual generation*.

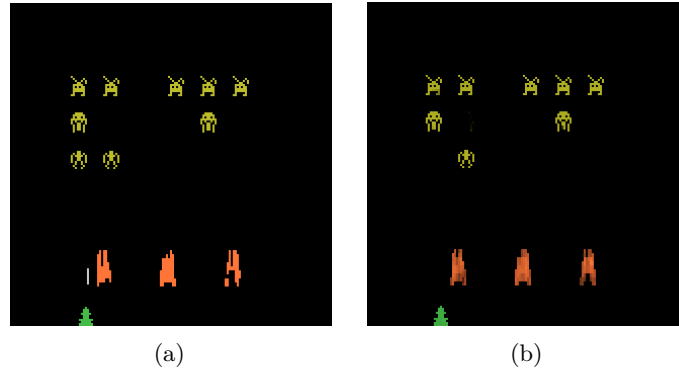


Figure 1.5: Counterfactual explanation of the agent’s behavior for the game *Space Invaders* [Olson et al., 2021]. (a) is the original observation, for which the agent selects the action “LEFT”, whereas (b) is the counterfactual observation, for which the agent selects the action “RIGHT_FIRE”.

Latent Space Usage for Counterfactual Generation

In this family of methods, **generative models** are used [Bond-Taylor et al., 2021] to exploit the Latent Representation (LR). This type of model learns to generate new data by imitating the training data distribution. In this context, generative models aim to produce alternative observations that are likely to alter the agent’s actions (see Figure 1.5) [Olson et al., 2021].

The main challenge of this approach lies in the fact that, to serve as explanations, counterfactuals must be both **close** and **feasible**. A close counterfactual refers to one that does not differ significantly from the original observation. A feasible counterfactual corresponds to an observation that is meaningful within the considered environment.

1.3.4 Agent Trajectory

Until now, we have presented methods that focus on explaining individual decisions made by the agent. We now consider methods that aim to provide insight into the agent’s strategy by analyzing its behavior over time.

The most direct approach consists in visualizing agent trajectories, most commonly through complete episodes. By analyzing these trajectories, the observer can infer the dynamics underlying the agent’s behavior and how its decisions evolve through time. This type of visualization is not specific to XRL. In practice, visualizing agent episodes is a standard step in the development and evaluation of RL. It provides a simple way to verify that the agent behaves as expected and to detect potential implementation errors.

This simple approach can be further enhanced using families of methods such as *hierarchical decomposition* and *extraction via critic analysis*.

Extraction via Critic Analysis

In this family of methods, the predictions of the critic are used to determine which **trajectories to present to the observer** [Sequeira and Gervasio, 2020].

A set of states is selected from multiple trajectories. For a given state, all possible actions are examined. The state is assigned a value equal to the difference between the critic’s prediction for the lowest-valued action and the prediction for the highest-valued action. Trajectories containing the states with the highest values according to this procedure are then chosen to be presented to the observer. These trajectories are considered **critical moments** of the agent and are therefore relevant for analysis. For this family of methods, the *selected trajectories* represent the interpretable information provided to the observer.

Hierarchical Decomposition

Hierarchical agents are composed of a set of **sub-policies** specialized for specific tasks within subsets of the environment’s states. This family of methods distributes the optimization of the reward function across multiple models, significantly simplifying the learning process. Moreover, this **decomposition** of the policy function can also provide explainability in the agent’s strategy [Beyret et al., 2019]. If a sub-policy is selected, it indicates that the agent will execute a specialized behavior. This behavioral specialization provides insight into the strategy the agent employs to maximize the reward function. The agent’s *trajectory*, together with the *sub-policy* selected for each action, is used as interpretable information.

1.3.5 Trajectory Prediction

Making predictions about the future of a trajectory allows one to form a representation of what the system expects, based on its training experience. These predictions are produced using value functions. Such functions provide insights into the future evolution of the agent’s trajectory. By their nature, this form of explanation informs the observer about the strategy adopted by the agent. Value functions provide an estimate of the expected cumulative reward from a given state. Consequently, a value function maps an observation to a scalar value in $\mathbb{R}$.

This aggregated prediction provides limited information about the underlying factors that contribute to this value. It does not directly reveal which aspects of the task are being optimized or how the predicted return is composed. To improve the interpretability of value-based explanations, a family of methods known as *critic enrichment* can be employed.

Critic Enrichment

In order to obtain interpretable information from the predictions of the **critic**, it is possible to **decompose** the critic into multiple **sub-functions** [Juozapaitis et al., 2019]. The reward function is thus decomposed into multiple sub-reward functions, each representing a **sub-objective** of the task to be accomplished. Similarly, for each of these sub-reward functions, corresponding sub-critic functions are defined, along with a function that combines their outputs. During training, the agent selects its actions so as to maximize this combination function of the sub-critics. Using this approach, for each observation and action, a prediction is obtained regarding the sub-objectives the agent is attempting to maximize (see Figure 1.6). This *prediction* is then used as the explanation for the observer.

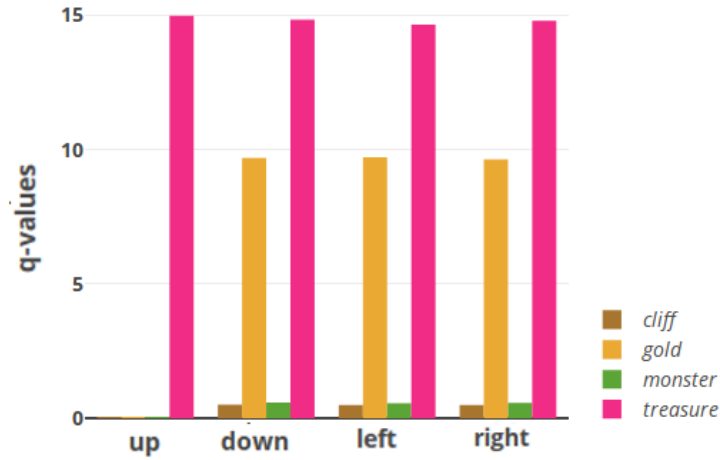


Figure 1.6: Prediction of sub-rewards as a function of the action selected by the agent for a given state [Juozaipaitis et al., 2019]. Depending on the action chosen by the agent, one can determine which sub-reward it aims to maximize.

1.3.6 Interaction Model

Modeling the interactions between an agent and its environment makes it possible to understand how the agent’s decisions are made and what impact they have on the environment. By incorporating these interactions into an explanatory model, one can better grasp the complex relationships between the agent and its environment, thereby identifying the determining factors in the decision-making process. By explicitly handling the agent–environment interaction, this type of explanation aims to explain the agent’s strategy. The families of methods used to obtain explanations in the form of agent–environment interactions are: *Bayesian network learning* and *LS usage for MDP generation*.

Bayesian Network Learning

The evolution of an agent within an environment can be observed as a set of **random variables** connected through a **Bayesian network** [Madumal et al., 2020]. Random variables are defined to represent both external aspects (the environment) and internal aspects of the agent (the actions). By analyzing trajectories, it becomes possible to infer causal relationships among these random variables. At the end of this process, a weighted causal graph is obtained, which serves as an explanation for the observer (see Figure 1.7). This *Bayesian network*, representing the interactions between the agent and the environment, is then used as the explanation.

Latent Space Usage for MDP Generation

Each layer of a NN can be viewed as a projection from one space to another [Zahavy et al., 2017]. As information propagates through successive layers of a NN, it becomes increasingly abstract. It can therefore be assumed that two vectors that are close in this projected space are also close in terms of the agent’s

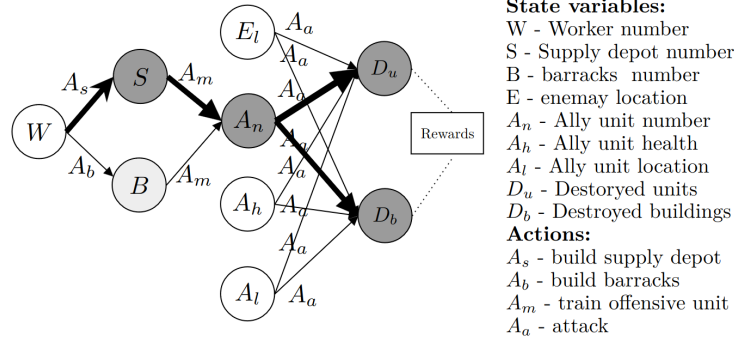


Figure 1.7: Agent-environment interaction model for the video game *StarCraft II* [Madumal et al., 2020]. Nodes represent random variables, and edges represent causal relationships between them.

perception. Based on this observation, it is possible to **redefine an MDP** by leveraging the **abstract representations** produced by the final layers of a NN. Once the new MDP has been constructed, a model of the interaction between the agent and its environment is obtained. This model, represented as a *graph*, is then used as the explanation.

Figure 1.8 illustrates this idea using a two-dimensional *t-SNE* projection of the LS of the NN used to represent the policy. Each point is colored according to the value function’s prediction for the corresponding state. An agreement can be observed between the structure of this projected space and the critic’s predictions: states that are assigned similar values by the critic also tend to be located close to one another in the projection. This agreement makes it possible to manually identify clusters of states, which likely correspond to states that are perceived as similar by the agent.

1.4 Conclusion

The taxonomy and the state-of-the-art review presented in this chapter make it possible to draw three main observations.

First, regarding the subject of explanation, Table 1.1 shows that every reviewed method targets either a **decision** or a **strategy**, and that none of them addresses the **learning process** itself, i.e., how the policy is progressively updated over the course of training. We identify two reasons for this absence. On the one hand, from a practical standpoint, this question is of limited interest: industrial applications generally deploy an already-trained policy, so the training procedure itself is rarely part of what needs to be explained once the agent is in production. On the other hand, training can itself be described as an iterative improvement of the policy. Explaining the learning process would therefore amount to explaining the sequence of policies produced across training iterations, which is conceptually close to explaining a strategy, only applied to the space of successive policies rather than to the space of successive states. Under this view, explaining a learning process does not constitute a fundamentally distinct problem but rather a variant of strategy explanation, which may account for

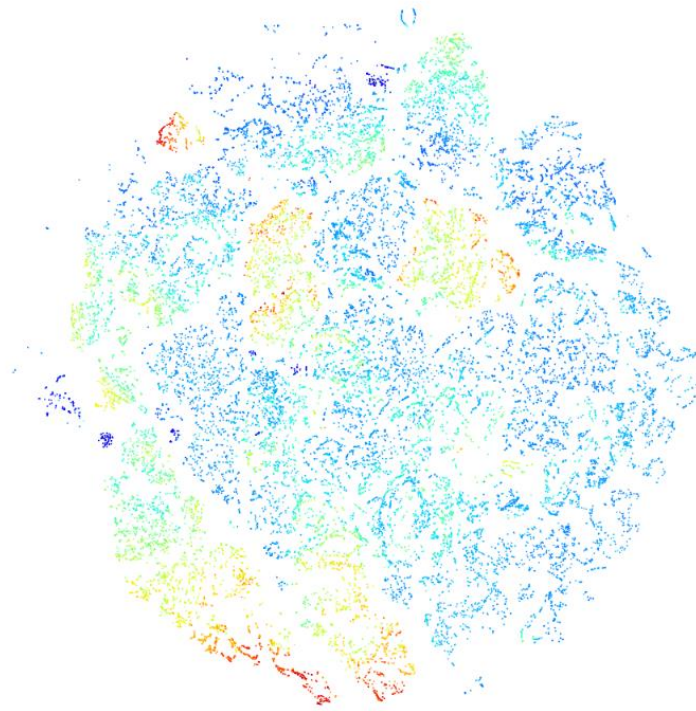


Figure 1.8: t -SNE projection of a neural network's latent space [Zahavy et al., 2017]. Each point corresponds to a state, and its color represents the value function's prediction for that state, with warmer colors indicating higher predicted values.

486 why it has not emerged as a separate subject in the literature.

487 Second, the diversity of explanation forms identified in this review (Table 1.1)
 488 reveals a limitation shared by nearly all the presented families of methods: the
 489 absence of reliable metrics to assess the impact of an explanation on the observer.
 490 Most works instead rely on subjective satisfaction questionnaires administered
 491 to human participants. This evaluation protocol is not fully satisfactory, since
 492 self-reported satisfaction is subject to numerous biases and does not necessarily
 493 reflect an actual improvement of the observer's mental model, as required by
 494 Definition 1.1.4.

495 Third, among the methods reviewed, *latent space usage for MDP generation*
 496 (Section 1.3.6) appears, in our view, the most promising direction. This method
 497 establishes a direct bridge between the deep representations learned by the agent
 498 and the search for concepts investigated in Chapter ??, applied here to the
 499 explanation of reinforcement learning agents. Rather than abstracting away or
 500 replacing the underlying deep learning support, as substitutive approaches do
 501 (Section 1.1), it exploits this support directly to produce an explanation, in line
 502 with the intrinsic explainability perspective adopted throughout this manuscript.
 503 However, as noted in Section 1.3.6, this method still relies on an informal, largely
 504 manual analysis of the projected latent space to identify clusters of states, which
 505 limits its rigor and its ability to systematically produce reliable explanations.

506 The next chapter builds on this approach and introduces a method designed to
507 address this limitation.

Bibliography

- [Barredo Arrieta et al., 2020] Barredo Arrieta, A., Díaz-Rodríguez, N., Del Ser, J., Bennetot, A., Tabik, S., Barbado, A., García, S., Gil-López, S., Molina, D., Benjamins, R., Chatila, R., and Herrera, F. (2020). Explainable artificial intelligence (xai): Concepts, taxonomies, opportunities and challenges toward responsible ai. Information Fusion, 58:82–115. **Article**.
- [Bellucci et al., 2021] Bellucci, M., Delestre, N., Malandain, N., and Zanni-Merk, C. (2021). Towards a terminology for a fully contextualized xai. Procedia Computer Science, 192:241–250. **Article**.
- [Beyret et al., 2019] Beyret, B., Shafti, A., and Faisal, A. A. (2019). Dot-to-dot: Explainable hierarchical reinforcement learning for robotic manipulation. In 2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS). IEEE. **Article**.
- [Bond-Taylor et al., 2021] Bond-Taylor, S., Leach, A., Long, Y., and Willcocks, C. G. (2021). Deep generative modelling: A comparative review of vaes, gans, normalizing flows, energy-based and autoregressive models. **Article**.
- [Chen et al., 2021] Chen, M., Tworek, J., Jun, H., Yuan, Q., de Oliveira Pinto, H. P., Kaplan, J., et al. (2021). Evaluating large language models trained on code. **Article**.
- [Doshi-Velez and Kim, 2017] Doshi-Velez, F. and Kim, B. (2017). Towards a rigorous science of interpretable machine learning. **Article**.
- [Garreau and von Luxburg, 2020] Garreau, D. and von Luxburg, U. (2020). Explaining the explainer: A first theoretical analysis of lime. In Chiappa, S. and Calandra, R., editors, Proceedings of the Twenty Third International Conference on Artificial Intelligence and Statistics, volume 108 of Proceedings of Machine Learning Research, pages 1287–1296. PMLR. **Article**.
- [Greydanus et al., 2018] Greydanus, S., Koul, A., Dodge, J., and Fern, A. (2018). Visualizing and understanding atari agents. **Article**. **Code**.
- [Guidotti et al., 2018] Guidotti, R., Monreale, A., Ruggieri, S., Turini, F., Giannotti, F., and Pedreschi, D. (2018). A survey of methods for explaining black box models. ACM Computing Surveys, 51(5):1–42. **Article**.
- [Gunning and Aha, 2019] Gunning, D. and Aha, D. (2019). Darpa’s explainable ai (xai) program. AI Magazine, 40(2):44–58. **Article**.

- [Huber et al., 2019] Huber, T., Schiller, D., and André, E. (2019). Enhancing explainability of deep reinforcement learning through selective layer-wise relevance propagation. In *KI 2019: Advances in Artificial Intelligence: 42nd German Conference on AI, Kassel, Germany, September 23–26, 2019, Proceedings 42*, pages 188–202. Springer. **Article**.
- [Itaya et al., 2021] Itaya, H., Hirakawa, T., Yamashita, T., Fujiyoshi, H., and Sugiura, K. (2021). Visual explanation using attention mechanism in actor-critic-based deep reinforcement learning. **Article**.
- [Juozapaitis et al., 2019] Juozapaitis, Z., Koul, A., Fern, A., Erwig, M., and Doshi-Velez, F. (2019). Explainable reinforcement learning via reward decomposition. In *IJCAI/ECAI Workshop on explainable artificial intelligence*. **Article**.
- [Lewis et al., 2021] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., tau Yih, W., Rocktäschel, T., Riedel, S., and Kiela, D. (2021). Retrieval-augmented generation for knowledge-intensive nlp tasks.
- [Madumal et al., 2020] Madumal, P., Miller, T., Sonenberg, L., and Vetere, F. (2020). Explainable reinforcement learning through a causal lens. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 2493–2500. **Article**.
- [Miller, 2019] Miller, T. (2019). Explanation in artificial intelligence: Insights from the social sciences. *Artificial Intelligence*, 267:1–38. **Article**.
- [Mnih et al., 2013] Mnih, V., Kavukcuoglu, K., Silver, D., Graves, A., Antonoglou, I., Wierstra, D., and Riedmiller, M. (2013). Playing atari with deep reinforcement learning.
- [Mott et al., 2019] Mott, A., Zoran, D., Chrzanowski, M., Wierstra, D., and Jimenez Rezende, D. (2019). Towards interpretable reinforcement learning using attention augmented agents. *Advances in neural information processing systems*, 32. **Article**.
- [Olah et al., 2020] Olah, C., Cammarata, N., Schubert, L., Goh, G., Petrov, M., and Carter, S. (2020). Zoom in: An introduction to circuits. *Distill*. **Article**.
- [Olson et al., 2021] Olson, M. L., Khanna, R., Neal, L., Li, F., and Wong, W.-K. (2021). Counterfactual state explanations for reinforcement learning agents via generative deep learning. *Artificial Intelligence*, 295:103455. **Article**. **Code**.
- [Payani and Fekri, 2019] Payani, A. and Fekri, F. (2019). Inductive logic programming via differentiable deep neural logic networks. **Article**. **Code**.
- [Rudin, 2019] Rudin, C. (2019). Stop explaining black box machine learning models for high stakes decisions and use interpretable models instead. *Nature Machine Intelligence*, 1(5):206–215. **Article**.
- [Sequeira and Gervasio, 2020] Sequeira, P. and Gervasio, M. (2020). Interestingness elements for explainable reinforcement learning: Understanding agents’ capabilities and limitations. *Artificial Intelligence*, 288:103367. **Article**.

- 583 [Seshia et al., 2022] Seshia, S. A., Sadigh, D., and Sastry, S. S. (2022). Toward
584 verified artificial intelligence. *Communications of the ACM*, 65(7):46–55.
585 **Article.**
- 586 [Sieusahai and Guzdial, 2021] Sieusahai, A. and Guzdial, M. (2021). Explaining
587 deep reinforcement learning agents in the atari domain through a surrogate
588 model. **Article.**
- 589 [Silver et al., 2017] Silver, D., Hubert, T., Schrittwieser, J., Antonoglou, I., Lai,
590 M., Guez, A., Lanctot, M., Sifre, L., Kumaran, D., Graepel, T., et al. (2017).
591 Mastering chess and shogi by self-play with a general reinforcement learning
592 algorithm. *arXiv preprint arXiv:1712.01815*.
- 593 [Simonyan et al., 2014] Simonyan, K., Vedaldi, A., and Zisserman, A. (2014).
594 Deep inside convolutional networks: Visualising image classification models
595 and saliency maps. **Article. Code.**
- 596 [Topin et al., 2021] Topin, N., Milani, S., Fang, F., and Veloso, M. (2021).
597 Iterative bounding mdps: Learning interpretable policies via non-interpretable
598 methods. In *Proceedings of the AAAI Conference on Artificial Intelligence*,
599 volume 35, pages 9923–9931. **Article.**
- 600 [Trivedi et al., 2022] Trivedi, D., Zhang, J., Sun, S.-H., and Lim, J. J. (2022).
601 Learning to synthesize programs as interpretable and generalizable policies.
602 **Article. Code.**
- 603 [Vaswani et al., 2023] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones,
604 L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2023). Attention is all you
605 need. **Article.**
- 606 [Weitkamp et al., 2019] Weitkamp, L., van der Pol, E., and Akata, Z. (2019).
607 Visual rationalizations in deep reinforcement learning for atari games. **Article.**
- 608 [Zahavy et al., 2017] Zahavy, T., Zrihem, N. B., and Mannor, S. (2017). Graying
609 the black box: Understanding dqns. **Article.**